

TARA CUDA TECHNICAL MANUAL

MADE BY:

Althea Zyrle M. Arceta, Antonio Gabriel N. Mendoza, Jose Tristan T. Tan

Adviser:

Mr. Roger Luis Uy

Overview

TARA CUDA is a Hybrid SIMT Implementation of Zhang's Six-Frame Algorithm written in CUDA C/C++, which contains both a sequential implementation and a parallel implementation of the algorithm. This document will discuss the relevant components of the parallel implementation. There are 3 modules included in this program: input, setup, and the main program loop. Some helper functions are also defined.

1. Input Module

a. FASTA Files

- The program will prompt the user to enter the filenames of the protein dataset and DNA input read sequences to be used. It uses the *readFastaSequences* and the *readFastaIDs* (for the proteins) functions and returns the arrays of sequences and IDs (for the proteins).

```
do {
    file_name.clear();
    cout << "Input file name for protein(exclude .fasta/.fastq):" << endl <<
"--> ";
    cin >> file_name;

    proteinInputs = readFastaSequences(file_name);
    proteinIdInputs = readFastaIDs(file_name);
    if (!proteinInputs.empty() && !proteinIdInputs.empty()) {
        break;
    }
} while (true);

do {
    file_name.clear();
    cout << "Input file name for DNA (exclude .fasta/.fastq):" << endl << "-->
";
    cin >> file_name;

    dnaInputs = readFastaSequences(file_name);
    if (!dnaInputs.empty()) {
        break;
    }
} while (true);
```

b. Setting Configuration

- The program will prompt the user to select the settings for the program:
 1. Mode: Sequential (0) or CUDA version (1)
 2. Frame Count: 3 or 6 Frames
 3. Top Score Display: Top 1 (0) or Top 5 (1) Scores

```
do {
    cout << "Choose mode [0 - Sequential] [1 - CUDA]:" << endl << "--> ";
    cin >> mode;

    if (mode == 0 || mode == 1)
        break;

    cout << "Invalid input please try again." << endl;
} while (true);

do {
    cout << "Choose frame count [3 - 3 Frame] [6 - 6 Frame]:" << endl << "--> ";
    cin >> frame;

    if (frame == 3 || frame == 6)
        break;

    cout << "Invalid input please try again." << endl;
} while (true);

do {
    cout << "Show Score [0 - Top 1] [1 - Top 5]:" << endl << "--> ";
    cin >> top;

    if (top == 0 || top == 1)
        break;

    cout << "Invalid input please try again." << endl;
} while (true);
```

2. Setup Module

a. BLOSUM62 Matrix

- The *readBlosum62* function reads the BLOSUM62.txt file and saves the matrix values into an array, which is then copied to device memory for use by the CUDA kernels.

```
void readBlosum62() {
    ifstream file("../Resource Files/BLOSUM62.txt");
    if (!file.is_open()) {
        cerr << "Error opening the BLOSUM62 file!" << endl;
        exit(1);
    }
    string skip;
```

```

getline(file, skip);
getline(file, skip);

char c = NULL;

    for (int i = 0; i < 24; i++) {
        file >> c;
        for (int j = 0; j < 24; j++) {
            file >> blosum62mat[i][j];
        }
    }
    file.close();
}
...

readBlosum62();
cudaMemcpyToSymbol(d_blosum62mat, blosum62mat, sizeof(blosum62mat), 0,
cudaMemcpyHostToDevice);

```

b. Protein to Int Map

- The *initialize_protein_to_int_map* function initializes a lookup map to translate amino acid characters to their corresponding integer indices in the BLOSUM62 matrix. This map is then copied to device memory for use by the CUDA kernels.

```

void initialize_protein_to_int_map() {

    vector<int> host_map(128, -1);

    host_map['A'] = 0;
    host_map['R'] = 1;
    host_map['N'] = 2;
    host_map['D'] = 3;
    host_map['C'] = 4;
    host_map['Q'] = 5;
    host_map['E'] = 6;
    host_map['G'] = 7;
    host_map['H'] = 8;
    host_map['I'] = 9;
    host_map['L'] = 10;
    host_map['K'] = 11;
    host_map['M'] = 12;
    host_map['F'] = 13;
    host_map['P'] = 14;
    host_map['S'] = 15;
    host_map['T'] = 16;
    host_map['W'] = 17;
    host_map['Y'] = 18;
    host_map['V'] = 19;
    host_map['B'] = 20;
    host_map['Z'] = 21;
    host_map['X'] = 22;
    host_map['U'] = 23;
}

```

```

    for (int i = 0; i < 128; ++i)
        prot_to_idx[i] = host_map[i];

    checkCudaErrors(cudaMemcpyToSymbol(d_prot_to_idx, host_map.data(),
sizeof(int) * host_map.size(), 0, cudaMemcpyHostToDevice));
}

```

3. Main Module

a. Loop Structure

- The main program utilizes a nested loop to iterate through the DNA and protein sequences. Each DNA input query is aligned against each sequence in the protein dataset.

```

for (int index_dna = 0; index_dna < dnaInputs.size(); index_dna++) {

    ...

    for (int index_prot = 0; index_prot < 4096; index_prot++) {

        ...

    }

    ...

}

```

b. Sequence Preparation and Reverse Complement

- The sequences to be aligned in the current iteration of the loop are obtained. To accomodate for six frame alignment, the reverse complement of the current DNA sequence is obtained,

```

DNA_sequence = dnaInputs[index_dna];
protein_sequence = proteinInputs[index_prot];
DNA_sequence_r = reverse_complement(DNA_sequence);

```

c. Matrix Allocation

- Matrices for scoring, insertion, and deletion with their corresponding traceback matrices are initialized. An additional holding matrix is used for the top score functionality. Another set of matrices are initialized for the reverse complement of the input.

```

int* sc_mat = (int*)malloc(size);
int* ins_mat = (int*)malloc(size);
int* del_mat = (int*)malloc(size);
int* sc_mat_hold = (int*)malloc(size);

int* t_sc_mat = (int*)malloc(size);

```

```

int* t_ins_mat = (int*)malloc(size);
int* t_del_mat = (int*)malloc(size);
int* t_sc_mat_hold = (int*)malloc(size);

int* sc_mat_r = (int*)malloc(size);
int* ins_mat_r = (int*)malloc(size);
int* del_mat_r = (int*)malloc(size);
int* sc_mat_hold_r = (int*)malloc(size);

int* t_sc_mat_r = (int*)malloc(size);
int* t_ins_mat_r = (int*)malloc(size);
int* t_del_mat_r = (int*)malloc(size);
int* t_sc_mat_hold_r = (int*)malloc(size);

```

d. Matrix Initialization

- The *init_local_v2* function initializes the first column and first four rows of the matrices to allow the recurrence relations to iterate through the matrices.

```

void init_local_v2(char* input_seq, char* ref_seq, int* sc_mat, int* ins_mat,
int* del_mat, int* t_sc_mat, int* t_ins_mat, int* t_del_mat, size_t N, size_t
M) {
    for (size_t i = 0; i < N; i++) {
        ins_mat[i * M + 0] = infn;
        t_ins_mat[i * M + 0] = infn;
    }

    for (size_t i = 0; i < N; i++) {
        for (size_t j = 0; j < M; j++) {
            if (i == 0 || j == 0) {
                sc_mat[i * M + j] = 0;
                t_sc_mat[i * M + j] = 0;
            }
        }
    }

    for (size_t j = 0; j < M; j++) {
        del_mat[0 * M + j] = infn;
        del_mat[2 * M + j] = infn;
        del_mat[3 * M + j] = infn;
        del_mat[1 * M + j] = sc_mat[0 * M + j] - gop - gep;

        t_del_mat[0 * M + j] = infn;
        t_del_mat[2 * M + j] = infn;
        t_del_mat[3 * M + j] = infn;
    }
}

```

```

        t_del_mat[1 * M + j] = 1;
    }

    for (size_t i = 4; i < N; i++) {
        del = del_mat[(i - 3) * M + 0] - gep;
        xscore = sc_mat[(i - 3) * M + 0] - gop - gep;

        del_mat[i * M + 0] = (del > xscore) ? del : xscore;
    }

    for (size_t i = 0; i < 4; i++) {
        for (size_t j = 1; j < M; j++) {
            insert = ins_mat[i * M + (j - 1)] - gep;
            xscore = sc_mat[i * M + (j - 1)] - gop - gep;
            prot_seq = d_DNA_to_Protein(input_seq, i - 1, i, i + 1);

            if (insert > xscore) {
                ins_mat[i * M + j] = insert;
                t_ins_mat[i * M + j] = 0;
            }
            else {
                ins_mat[i * M + j] = xscore;
                t_ins_mat[i * M + j] = 1;
            }

            insert = ins_mat[i * M + j];
            del = del_mat[i * M + j];

            if (i == 1) {
                xscore = sc_mat[0 * M + (j - 1)] + score(prot_seq, ref_seq[j -
1]);

                if (insert >= del && insert >= xscore) {
                    sc_mat[i * M + j] = insert;
                    t_sc_mat[i * M + j] = -2;
                }
                else if (del >= insert && del >= xscore) {
                    sc_mat[i * M + j] = del;
                    t_sc_mat[i * M + j] = -1;
                }
                else {
                    sc_mat[i * M + j] = xscore;
                    t_sc_mat[i * M + j] = 1;
                }
            }
            else if (i == 2) {

```

```

        xscore = sc_mat[0 * M + (j - 1)] + score(prot_seq, ref_seq[j -
1]) - shift;

        if (insert > xscore) {
            sc_mat[i * M + j] = insert;
            t_sc_mat[i * M + j] = -2;
        }
        else {
            sc_mat[i * M + j] = xscore;
            t_sc_mat[i * M + j] = 2;
        }
    }
    else if (i == 3) {
        xscore = sc_mat[1 * M + (j - 1)] + score(prot_seq, ref_seq[j -
1]) - shift;

        if (insert > xscore) {
            sc_mat[i * M + j] = insert;
            t_sc_mat[i * M + j] = -2;
        }
        else {
            sc_mat[i * M + j] = xscore;
            t_sc_mat[i * M + j] = 2;
        }
    }

    if (sc_mat[i * M + j] < 0)
        sc_mat[i * M + j] = 0;
}
}
}

```

e. CUDA Matrix Scoring

- In performing matrix scoring with a Hybrid CUDA implementation, several additional preparatory steps are required.
- i. Grid Block Configuration
 - Block size was set to 1024 (32 x 32) to maximize the size of each kernel. Grid size is dynamically determined in each iteration of the host-side loop, calculating the number of submatrices in the current antidiagonal.

```

unsigned int curr_diag_blocks = 0;
if (last_sub_y_curr_diag >= first_sub_y_curr_diag)
    curr_diag_blocks = last_sub_y_curr_diag - first_sub_y_curr_diag + 1;
dim3 currGridDim(1, 1);

if (curr_diag_blocks > 0)
    currGridDim = dim3(curr_diag_blocks, 1);

```

ii. Memory Management

- Data required needs to be accessible on the device, and thus memory on the device needs to be allocated using *cudaMalloc*. *cudaMemcpy* is used to transfer the initialized data. *cudaMemset* is used to ensure the rest of the matrices are properly initialized to 0.

```
checkCudaErrors(cudaMalloc(&d_DNA_sequence, N_size));
checkCudaErrors(cudaMalloc(&d_protein_sequence, M_size));
checkCudaErrors(cudaMalloc(&d_DNA_sequence_r, N_size));
checkCudaErrors(cudaMemcpy(d_DNA_sequence, c_DNA_sequence, N_size,
cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_protein_sequence, c_protein_sequence, M_size,
cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_DNA_sequence_r, c_DNA_sequence_r, N_size,
cudaMemcpyHostToDevice));

checkCudaErrors(cudaMalloc(&d_sc_mat, size));
checkCudaErrors(cudaMalloc(&d_ins_mat, size));
checkCudaErrors(cudaMalloc(&d_del_mat, size));

checkCudaErrors(cudaMalloc(&d_t_sc_mat, size));
checkCudaErrors(cudaMalloc(&d_t_ins_mat, size));
checkCudaErrors(cudaMalloc(&d_t_del_mat, size));
...

checkCudaErrors(cudaMemset(d_sc_mat, 0, size));
checkCudaErrors(cudaMemset(d_ins_mat, 0, size));
checkCudaErrors(cudaMemset(d_del_mat, 0, size));
checkCudaErrors(cudaMemset(d_t_sc_mat, 0, size));
checkCudaErrors(cudaMemset(d_t_ins_mat, 0, size));
checkCudaErrors(cudaMemset(d_t_del_mat, 0, size));
...

checkCudaErrors(cudaMemcpy(d_sc_mat, sc_mat, size, cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_ins_mat, ins_mat, size, cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_del_mat, del_mat, size, cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_t_sc_mat, t_sc_mat, size,
cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_t_ins_mat, t_ins_mat, size,
cudaMemcpyHostToDevice));
checkCudaErrors(cudaMemcpy(d_t_del_mat, t_del_mat, size,
cudaMemcpyHostToDevice));
...
```

iii. Host-Side Loop

- The program uses a host-side loop that launches the kernels along the antidiagonals. CUDA streams are used for concurrent launching of the forward and reverse complement strand kernels.

```
for (unsigned int diag = 0; diag < numSubmatrixRows + numSubmatrixCols - 1; ++diag)
```



```

{
    unsigned int first_sub_y_curr_diag = std::max(0, (int)diag -
        (int)(numSubmatrixCols - 1));
    unsigned int last_sub_y_curr_diag = std::min((int)diag, (int)numSubmatrixRows -
        1);

    //Grid dimension calculation

    scoring_local_v2_cuda <<<GridDim, blockDim, 0, stream1 >>> (...);
    scoring_local_v2_cuda <<<GridDim, blockDim, 0, stream2 >>> (...);
}

```

iv. CUDA Kernel

- The CUDA kernel is responsible for calculating the insertion, deletion, and substitution scores and updating the respective matrices in place. In addition to the scores, traversal values that will be used in the traceback are also computed and stored. The kernel also utilizes a conditional anti-diagonal loop to ensure that only cells within the current anti-diagonal loop are calculated. Block synchronization calls are performed in every iteration of the loop to guarantee that threads within the anti-diagonal wait for each other and respect the data dependencies, preventing them from performing calculations with values that are not yet available. Grid-stride calculations are performed within the kernel to allow a single kernel to perform the scoring for all tiles in the current anti-diagonal.

```

__global__ void scoring_local_v2_cuda(...) {
    cg::thread_block block = cg::this_thread_block();

    unsigned int diagBlockIdx = blockIdx.x;

    unsigned int submatrixStartY = currDiagFirstSubY + diagBlockIdx;
    unsigned int submatrixStartX = currDiag - submatrixStartY;

    int i = submatrixStartY * blockDim.y + threadIdx.y;
    int j = submatrixStartX * blockDim.x + threadIdx.x;

    int insert = 0;
    int del = 0;
    int xscore = 0;
    int sc_1 = 0, sc_2 = 0, sc_3 = 0;
    int scoring = 0;
    char prot_seq;

    if (i >= 4 && i < N && j >= 1 && j < M) {
        for (unsigned int sub_diag_in_tile = 0; sub_diag_in_tile < 2 *
submatrixSide - 1; ++sub_diag_in_tile) {
            if (((i - (submatrixStartY * blockDim.y)) + (j -
(submatrixStartX * blockDim.x))) == sub_diag_in_tile) {

```

```

        if (i == N - 1) {
            u_ins_mat[i * M + j] = infn;
            u_sc_mat[i * M + j] = 0;
            u_t_sc_mat[i * M + j] = infn;
        }
        else {
            prot_seq = d_DNA_to_Protein(input_seq, i - 1, i, i +
1);

            scoring = d_score(prot_seq, ref_seq[j - 1]);
            insert = u_ins_mat[i * M + (j - 1)] - c_gep;
            xscore = u_sc_mat[i * M + (j - 1)] - c_gop - c_gep;

            u_ins_mat[i * M + j] = (insert > xscore) ? insert :
xscore;

            u_t_ins_mat[i * M + j] = (insert > xscore) ? 0 : 1;

            del = u_del_mat[(i - 3) * M + j] - c_gep;
            xscore = u_sc_mat[(i - 3) * M + j] - c_gop - c_gep;

            u_del_mat[i * M + j] = (del > xscore) ? del : xscore;
            u_t_del_mat[i * M + j] = (del > xscore) ? 0 : 1;

            insert = u_ins_mat[i * M + j];
            del = u_del_mat[i * M + j];
            sc_1 = u_sc_mat[(i - 2) * M + (j - 1)] + scoring -
c_shift;

            sc_2 = u_sc_mat[(i - 3) * M + (j - 1)] + scoring;
            sc_3 = u_sc_mat[(i - 4) * M + (j - 1)] + scoring -
c_shift;

            int max_current_score;
            int trace_value;

            max_current_score = (insert >= del) ? insert : del;
            trace_value = (insert >= del) ? -2 : -1;

            if (sc_2 > max_current_score) {
                max_current_score = sc_2;
                trace_value = 3;
            }
            if (sc_1 > max_current_score) {
                max_current_score = sc_1;
                trace_value = 2;
            }
            if (sc_3 > max_current_score) {
                max_current_score = sc_3;
                trace_value = 4;
            }

            u_sc_mat[i * M + j] = (max_current_score < 0) ? 0 :
max_current_score;

            u_t_sc_mat[i * M + j] = trace_value;
        }
    }
    cg::sync(block);

```

```

    }
}
}

```

f. Traceback and Output

- Shows the protein name and sequence number where it matched, displays the possible frames for the DNA query, shows the alignment score, and shows the index where the protein matched to the DNA query.

```

if (mode == 0 && top_hold == top_scores[i]) {
    cout << proteinIdInputs[top_indexes[i]] << endl << "Sequence: " <<
top_indexes[i] << endl;
    QueryPerformanceCounter(&start);
    scoring_local_v2(c_DNA_sequence, c_protein_sequence, sc_mat, ins_mat,
del_mat, t_sc_mat, t_ins_mat, t_del_mat, N, M);
    QueryPerformanceCounter(&end);
    double elapsed1 = (end.QuadPart - start.QuadPart) * 1000.0 /
freq.QuadPart;

    traceV2_ld_check(c_DNA_sequence, c_protein_sequence, sc_mat, t_sc_mat, N,
M, top_indexes[i], index);

    if (frame == 3) {
        traceV2_ld(c_DNA_sequence, c_protein_sequence, sc_mat, t_sc_mat, N, M,
top_indexes[i], index);
        cout << endl << "Score: " << top_scores[i] << endl;
        cout << "Start to End match in Protein: " <<
myArray[top_indexes[i]][1] << "-" << myArray[top_indexes[i]][2] << endl <<
endl;
        cout << "Time in ms: " << elapsed1 << endl << endl;
    }

    if (frame == 6) {
        QueryPerformanceCounter(&start);
        scoring_local_v2(c_DNA_sequence_r,
c_protein_sequence, sc_mat_r, ins_mat_r, del_mat_r, t_sc_mat_r, t_ins_mat_r,
t_del_mat_r, N, M);
        QueryPerformanceCounter(&end);
        double elapsed = (end.QuadPart - start.QuadPart) * 1000.0 /
freq.QuadPart;

        traceV2_ld_check(c_DNA_sequence, c_protein_sequence, sc_mat_r,
t_sc_mat_r, N, M, top_indexes[i], index_r);
    }
}

```

```

        if (index[0] >= index_r[0]) {
            traceV2_ld(c_DNA_sequence, c_protein_sequence, sc_mat, t_sc_mat,
N, M, top_indexes[i], index);
            cout << endl << "Score: " << top_scores[i] << endl;
            cout << "Start to End match in Protein: " << index[3] << "-" <<
index[2] << endl << endl;
            cout << "Time in ms: " << elapsed1 << endl << endl;
        }
        else {
            cout << "Reverse: " << endl;
            traceV2_ld(c_DNA_sequence_r, c_protein_sequence, sc_mat_r,
t_sc_mat_r, N, M, top_indexes[i], index_r);
            cout << endl << "Score: " << top_scores[i] << endl;
            cout << "Start to End match in Protein: " << index_r[3] << "-" <<
index_r[2] << endl << endl;
            cout << "Time in ms: " << elapsed << endl << endl;
        }
    }
}

```

4. Helper Functions

a. readFastaSequences

- Reads the FASTA file and converts it into an array of strings.

```

vector<string> readFastaSequences(const string& filename) {
    ifstream file("./Resource Files/" + filename + ".fasta");
    vector<string> sequences;
    string line, sequence;

    if (!file) {
        cerr << "Error: Unable to open file " << filename << endl;
        return sequences;
    }

    while (getline(file, line)) {
        if (line.empty()) continue;
        if (line[0] == '>') {
            if (!sequence.empty()) {
                sequences.push_back(sequence);
                sequence.clear();
            }
        }
        else {
            sequence += line;
        }
    }

    if (!sequence.empty()) {
        sequences.push_back(sequence);
    }
}

```

```

    }

    file.close();

    return sequences;
}

```

b. readFastaIDs

- Reads the FASTA file and obtains the protein ID of the sequences.

```

vector<string> readFastaIDs(const string& filename) {
    ifstream file("./Resource Files/" + filename + ".fasta");
    vector<string> sequenceIDs;
    string line;

    if (!file) {
        cerr << "Error: Unable to open file " << filename << endl;
        return sequenceIDs;
    }

    while (getline(file, line)) {
        if (!line.empty() && line[0] == '>') {
            sequenceIDs.push_back(line.substr(1)); // Remove '>' and store
only the ID
        }
    }

    return sequenceIDs;
}

```

c. checkCudaErrors

- In order to ensure proper CUDA programming, a CUDA error checking utility macro was implemented. The macro wraps individual CUDA API calls to catch errors and halt execution, avoiding incorrect CUDA calls.

```

#define checkCudaErrors(val) check( (val), #val, __FILE__, __LINE__)
template<typename T>
void check(T err, const char* const func, const char* const file, const int
line)
{
    if (err != cudaSuccess) {
        fprintf(stderr, "CUDA error at: %s : %d\n", file, line);
        fprintf(stderr, "%s %s\n", cudaGetErrorString(err), func);
        exit(1);
    }
}

```